

CloudServe Support Copilot — Complete Verbatim Presentation Speech

How to use this document:

Read this text out loud word-for-word during your presentation. Follow the bracketed cues like [CLICK TO SLIDE X] and [DEMO ACTION] to sync your presentation slides and live UI demo.

■ Verbatim Speech Script (20 Minutes Total)

Section 1: Introduction & Executive Summary (0:00 - 1:00)

■ [CLICK TO SLIDE 1: Title & Executive Summary]

"Good morning, esteemed members of the evaluation panel, faculty, and colleagues.

Today, I am excited to present our Capstone project: **CloudServe Support Copilot** — a production-grade Multimodal Retrieval-Augmented Generation system and Agentic Triage Workbench.

In modern cloud platforms, technical support teams face a major operational hurdle: handling thousands of complex customer tickets arriving daily across Email, Chat, and Community Forums. Support engineers spend up to 70% of their time manually digging through long PDF manuals, Word guides, and Excel spreadsheets just to find answers to recurring technical questions.

Our system automates Tier 1 support triage by combining multi-format document parsing, 384-dimensional vector embeddings, hybrid semantic and keyword retrieval, Gemini 2.5 Flash LLM synthesis, and a Human-in-the-Loop Angular workbench.

With this architecture, we reduced average first response times from **10 hours down to 1.8 seconds**, increased First Contact Resolution to **68.5%**, and eliminated LLM hallucinations through verifiable, page-level source citations.

Let me take you through how we built it."

Section 2: Industry Problem & Technical Challenges (1:00 - 2:30)

■ [CLICK TO SLIDE 2: Enterprise Challenge & Problem Statement]

"Before designing our architecture, we identified four critical technical bottlenecks in traditional enterprise support:

First, **Information Fragmentation Across Formats**. Critical support knowledge doesn't live in plain text files alone. It is spread across multi-page PDFs, Word user guides, complex Excel billing spreadsheets, and architectural diagrams. Traditional text chunkers strip away table headers and destroy structural context.

Second, **The Risk of LLM Hallucinations**. In technical support, a language model cannot be allowed to invent non-existent CLI flags, promise invalid refunds, or quote wrong API parameters. Unconstrained AI

models create serious compliance and financial risks.

Third, **High SLA Violations**. Manual ticket triage creates massive response queues, leading to SLA breaches and low customer satisfaction.

And fourth, **Context Loss During Escalations**. When a Tier 1 agent escalates a ticket to a Tier 2 specialist, previous search context is lost, forcing the senior engineer to start the investigation from scratch.

Our system was specifically engineered to solve these four challenges."

Section 3: End-to-End System Architecture (2:30 - 4:30)

■ [CLICK TO SLIDE 3: System Architecture Flow]

"To tackle these challenges, we designed a four-layered modular system architecture, as shown on screen.

Layer 1 is our **Knowledge Source and Document Ingestion Layer**. It connects to Google Drive, ingesting PDFs, Word documents, Excel spreadsheets, and visual diagrams, converting them into 384-dimensional dense vectors using SentenceTransformers.

Layer 2 is our **FastAPI Core Engine** built in Python. It exposes high-speed REST endpoints for intent classification, hybrid retrieval, reranking, and metrics aggregation.

Layer 3 is our **Generative & Safety Layer**. It uses Gemini 2.5 Flash for grounded synthesis, protected by a strict 3-second thread pool timeout safeguard and an in-memory LRU response cache that delivers repeat queries in just 5.6 milliseconds.

Finally, Layer 4 is our **Human-in-the-Loop Angular 18 Workbench**. This is where support agents review inbound tickets, edit AI drafts inline, and execute automated decision logging directly to Supabase PostgreSQL.

Let's look at how data flows through each layer."

Section 4: Multi-Modal Knowledge Base Ingestion (4:30 - 6:30)

■ [CLICK TO SLIDE 4: Multi-Modal Ingestion Pipeline]

"Data ingestion begins at Layer 1 with **Structure-Aware Parsing**.

When a document is fetched from Google Drive, our parser detects its MIME type. For PDFs and Word documents, it preserves heading hierarchies, section titles, and page numbers.

For Excel spreadsheets, standard text splitters break rows and columns apart. Our parser serializes row-column matrices into Markdown formatted tables, ensuring that table headers remain attached to cell values.

Each extracted chunk is converted into a 384-dimensional dense vector using the `all-MiniLM-L6-v2` model. These vectors are inserted into Supabase PostgreSQL, indexed using an **HNSW — Hierarchical Navigable Small World** — vector index configured with `vector_cosine_ops`. This enables sub-10-millisecond cosine similarity searches across millions of vector chunks."

Section 5: Hybrid Retrieval & Reranking Engine (6:30 - 8:30)

■ [CLICK TO SLIDE 5: Hybrid Search & RRF Reranking]

"A major flaw with relying solely on vector similarity search is that it can miss exact technical terms — such as specific configuration keys like `initialDelaySeconds` or status codes like `HTTP 401`.

To solve this, we implemented a **Dual-Path Hybrid Retrieval Engine**.

When a query arrives, our system executes two parallel searches:

1. A **Dense Vector Semantic Search** via Supabase pgvector RPC `match_document_chunks` to capture conceptual intent.
2. A **Sparse Keyword Search** via a local BM25 engine to capture exact technical term matches.

We then combine these two result streams using **Reciprocal Rank Fusion — or RRF** — using the mathematical formula shown on screen. RRF scores each chunk based on its rank in both dense and sparse lists. Chunks that rank high in both semantic meaning and exact term matching are elevated to the top.

The top 5 reranked passages are formatted with document IDs, page numbers, and section titles before being passed to the generation engine."

Section 6: Grounded LLM Generation & Safety Guardrails (8:30 - 10:30)

■ [CLICK TO SLIDE 6: Grounded Generation & Guardrail Architecture]

"Moving to generation, our top priority is **Zero-Hallucination Accuracy**.

Our prompt construction forces the LLM to answer strictly using the retrieved context blocks. If the context does not contain enough evidence, the model is instructed to state explicitly: *'The retrieved documents do not contain enough information to answer this question.'*

Every assertion in the generated answer includes inline bracketed citations — [1], [2], [3] — linking directly to the source document name, page number, and section header.

To ensure enterprise reliability, we built two critical performance safeguards:

First, a **3-second ThreadPoolExecutor Timeout Safeguard**. If an external network API call hangs or fails, the system automatically switches to our fast, deterministic local synthesis engine.

Second, an **In-Memory LRU Response Cache** in our FastAPI router that serves repeat lookups in **5.61 milliseconds**."

Section 7: Human-in-the-Loop Angular Workbench (10:30 - 12:30)

■ [CLICK TO SLIDE 7: Support Copilot Workbench UI]

"Now let's examine the user experience on our Angular 18 Workbench.

We firmly believe that AI in enterprise support should **augment human agents, not replace them blindly**.

Our workbench provides a clean glassmorphism interface. The left sidebar displays inbound customer tickets, categorized by channel — Email, Chat, or Forum — and urgency level.

When an agent selects a ticket, the copilot automatically runs hybrid search, generates a grounded draft answer, and displays the verifiable source citations.

The agent has complete control:

- They can click **'Edit Draft'** to customize the response inline in an interactive text area.
- They can click **'Approve & Send'** to dispatch the approved response to the customer.
- Or, if the issue requires senior intervention, they can click **'Escalate to Tier 2'**, which automatically packages the search context and logs an escalation audit record in Supabase."

Section 8: Live System Demonstration (12:30 - 15:00)

■ [CLICK TO SLIDE 8: Live System Demonstration]

"Now, I will conduct a live demonstration of our platform operating on `http://localhost:4200`.

■ [DEMO ACTION 1: Click on Ticket 1 in Workbench Left Sidebar]

Here, we have Ticket 1001 from customer Alex Rivera, asking about container liveness probe delays. As I select it, the copilot executes hybrid search against Supabase pgvector and returns a grounded answer citing Page 3 of `CloudServe_Platform_Overview.pdf`, explaining the `initialDelaySeconds: 45` parameter.

■ [DEMO ACTION 2: Click on Ticket 2 in Workbench Left Sidebar]

Next, Ticket 1002 from Sarah Chen reports an HTTP 401 Unauthorized error. The copilot retrieves Page 2 of `API_Authentication_Guide.docx`, identifying that the API key was passed in body parameters instead of the required `Authorization: Bearer` header.

■ [DEMO ACTION 3: Click 'Edit Draft' button on Ticket 2]

As an agent, I can click 'Edit Draft', modify the response text directly, and click 'Save Draft Changes'. Notice the instant toast notification banner confirming the update.

■ [DEMO ACTION 4: Click 'Escalate to Tier 2' button on Ticket 3]

Finally, Ticket 1003 from Marcus Brody requests a billing refund for idle nodes. The copilot retrieves the Excel billing policy sheet and recommends escalation. As I click 'Escalate to Tier 2', the system updates the ticket status badge to 'Escalated', logs an audit entry in Supabase `decision_logs`, and expands the Escalation Context Package Drawer for Tier 2 handoff."

Section 9: Governance, Metrics & Audit Persistence (15:00 - 16:30)

■ [CLICK TO SLIDE 9: Governance & SLA Analytics]

"[CLICK TO ANALYTICS TAB IN SPA]

Beyond ticket triage, governance and observability are built into every level of our platform.

In our Analytics dashboard, support managers track real-time performance indicators:

- **Average Response Time** dropped from 10 hours to **1.8 seconds**, representing a 97% speed improvement.
- **First Contact Resolution Rate** reached **68.5%**, surpassing our target of 65%.
- **SLA Compliance Rate** stands at **98.2%**.

Every approval, inline edit, and Tier 2 escalation is immutably written to Supabase PostgreSQL table `public.decision_logs`, recording agent IDs, timestamps, confidence scores, and guardrail audit flags for regulatory compliance."

Section 10: Conclusion & Panel Q&A; (16:30 - 20:00)

■ [CLICK TO SLIDE 10: Conclusion & Q&A;]

"In conclusion, the **CloudServe Support Copilot** demonstrates how modern Multimodal RAG, hybrid search, grounded LLM prompting, and human-in-the-loop UI design come together to deliver enterprise-grade support automation.

Our system is fast, verifiable, zero-hallucination, and fully tested with a 100% pass rate across all automated unit and integration test suites.

Thank you very much for your time and attention. I am now ready to take your questions."

■ Panel Q&A; Cheatsheet (Read when asked by panelists)

If Panelist asks: **"How do you guarantee that the LLM won't hallucinate fake information?"**

Read this response:

"We enforce zero hallucinations through three strict mechanisms:

First, our system prompt explicitly forbids using outside knowledge and forces the model to state 'insufficient information' if context is missing.

Second, every generated statement must include bracketed citations pointing to exact document names, page numbers, and section headers.

Third, our Human-in-the-Loop design requires a Tier 1 agent to review and approve or edit all draft answers before sending."

If Panelist asks: **"Why did you use Hybrid Search instead of just Vector Search?"**

Read this response:

"Dense vector search is great for semantic meaning, but can overlook exact string matches like code variables, CLI parameters, or status codes — such as `initialDelaySeconds` or `HTTP 401`. BM25 guarantees exact keyword matching, and Reciprocal Rank Fusion combines both so that results high in both semantic intent and keyword accuracy rank top."

If Panelist asks: *"How do you handle complex tables from Excel spreadsheets?"*

Read this response:

"Instead of splitting raw text by character counts, our multi-format parser converts spreadsheet rows and columns into Markdown matrix format. This preserves column headers alongside cell data so the spatial context of financial or billing metrics is maintained."

If Panelist asks: *"What happens if the Gemini API goes down or becomes slow?"*

Read this response:

"We implemented a 3-second thread pool timeout safeguard and API key validation. If the external Gemini API times out or fails, our engine automatically falls back to our local deterministic grounded synthesis generator, ensuring the workbench never freezes or hangs."